DEEP LEARNING SERIES

Recurrent Neural Networks (RNN)

Sequence Modeling, Memory Mechanisms & Applications

Full Form	Recurrent Neural Network
Primary Use	Sequential / time-series data
Key Idea	Hidden state that carries information across time
Famous Variants	LSTM, GRU, Bidirectional RNN, Seq2Seq
Main Challenge	Vanishing / exploding gradients

Educational Overview · August 2026

1. What is a Recurrent Neural Network?

A Recurrent Neural Network (RNN) is a class of neural networks designed to process sequential data. Unlike feed-forward networks that assume all inputs are independent, RNNs maintain a hidden state (memory) that is updated at every time step. This allows the network to capture temporal dependencies — information from earlier elements in the sequence can influence the processing of later elements.

At each time step t the network receives an input vector x_t and the previous hidden state h_{t-1}. It then computes a new hidden state h_t and (optionally) an output y_t. The same set of weights is reused at every time step, which keeps the number of parameters independent of sequence length.

Why Sequence Modeling Matters

Many real-world signals are ordered in time or space: natural language, speech, stock prices, sensor readings, DNA sequences, and video frames. A model that cannot remember earlier context will fail on tasks such as language modeling, machine translation, or predicting the next value in a time series. RNNs were the first widely successful architecture that could, in principle, learn long-range dependencies.

Basic Recurrence Equation

In the simplest (“vanilla”) RNN the hidden state is updated by a linear transformation of the current input and the previous hidden state, followed by a non-linearity (usually tanh or ReLU):

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$$

The same weight matrices W_{xh} and W_{hh} are shared across all time steps. This parameter sharing is what makes RNNs able to generalize to sequences of different lengths.

2. The Vanishing & Exploding Gradient Problem

When training an RNN with back-propagation through time (BPTT), gradients are multiplied by the recurrent weight matrix at every time step. If the largest eigenvalue of that matrix is less than 1, gradients shrink exponentially and effectively vanish for long sequences. If it is greater than 1, gradients explode. Either way, learning long-term dependencies becomes extremely difficult for a plain RNN.

Long Short-Term Memory (LSTM)

LSTM, introduced by Hochreiter & Schmidhuber in 1997, solves the vanishing-gradient problem with a carefully designed memory cell and three gating mechanisms:

- **Forget gate** — decides what information to discard from the cell state.
- **Input gate** — decides which new information to store in the cell state.
- **Output gate** — controls what parts of the cell state are exposed as the hidden state.

The cell state itself is updated with additive interactions rather than multiplicative ones, allowing gradients to flow more freely across many time steps. LSTMs became the dominant architecture for sequence tasks throughout the 2010s.

Gated Recurrent Unit (GRU)

The GRU, proposed in 2014, is a simplified variant of the LSTM that combines the forget and input gates into a single “update gate” and merges the cell state with the hidden state. It has fewer parameters than an LSTM and often performs comparably, making it popular when computational resources are limited.

3. Common RNN Architectures & Training

Bidirectional RNNs

In many tasks (for example named-entity recognition or speech recognition) the correct decision at a given position depends on both past and future context. A bidirectional RNN runs two independent recurrent networks — one forward and one backward — and concatenates their hidden states. This gives every time step a complete view of the sequence.

Encoder–Decoder (Seq2Seq)

Sequence-to-sequence models use one RNN (the encoder) to compress an input sequence into a fixed-length vector (or a sequence of vectors) and another RNN (the decoder) to generate an output sequence from that representation. This framework powered early neural machine translation systems and is still used for tasks such as summarization and dialogue.

Teacher Forcing & Scheduled Sampling

During training of sequence generators it is common to feed the ground-truth previous token into the decoder (teacher forcing). At inference time the model must use its own predictions, creating a train–test mismatch. Scheduled sampling gradually replaces ground-truth tokens with model predictions to bridge this gap.

Gradient Clipping

Even with LSTMs and GRUs, gradients can still occasionally explode. A simple and effective remedy is gradient clipping: if the norm of the gradient vector exceeds a threshold, the whole vector is scaled down so that its norm equals the threshold. This stabilizes training without changing the direction of the update.

4. Applications of RNNs & Their Successors

- **Language modeling & text generation** — predicting the next word or character.
- **Machine translation** — early neural MT systems were based on seq2seq RNNs.
- **Speech recognition** — converting audio frames into text transcripts.
- **Time-series forecasting** — stock prices, weather, sensor data, energy demand.
- **Named-entity recognition & part-of-speech tagging** — sequence labeling in NLP.
- **Music generation & handwriting synthesis** — modeling continuous sequential data.
- **Video analysis** — action recognition and captioning when combined with CNNs.

From RNNs to Transformers

Although Transformers (based on self-attention) have largely replaced RNNs as the default architecture for large-scale language modeling, RNNs and especially LSTMs remain highly relevant. They are still preferred when latency, memory footprint, or streaming (online) processing is critical, and they continue to serve as strong baselines and components inside hybrid models.

5. Summary & Key Takeaways

Recurrent Neural Networks introduced the idea of a persistent hidden state that carries information across time steps, making them naturally suited to sequential data. The vanishing-gradient problem limited early vanilla RNNs, but LSTM and GRU cells solved it with gating mechanisms that allow long-term memory. Bidirectional processing, encoder–decoder frameworks and careful training techniques further expanded their capabilities.

Even in the age of Transformers, understanding RNNs remains valuable: they illustrate fundamental concepts of sequential computation, parameter sharing over time, and the importance of gradient flow. For many practical problems — especially those requiring low latency or continuous streaming — RNNs and LSTMs continue to be excellent, efficient solutions.

End of document · RNN Overview · 5 pages